

Day0400 -- IoTs Platform and SCADA System

เจ้าของงาน ID: B6728960 Name:มงคล น้อยบาท

ผู้ทดลอง ID: B6729615 Name:ปภังกร สมสุข

- Download >>
https://drive.google.com/drive/folders/1gLpJZLV8fjLuK5wv41XhhycfM2eyHfGh?usp=drive_link
- ทำการทดลอง เก็บข้อมูล ลองคนต่อกลุ่ม แล้วแยกกันเติมรายละเอียดรายงานให้เรียบร้อย
- กำหนดชื่อไฟล์ตามรูปแบบ "B3601234 - Wichai Srisuruk – Day0400 IoTs and SCADA.pdf"
- ส่งแบบ pdf รอบโครงสร้าง ก่อน 20260815-1700 ที่
<https://forms.gle/MWPNSayVonMBjEdn8>
- ส่งแบบ pdf รอบสมบูรณ์ ก่อน 20260830-0600 ที่
<https://forms.gle/MWPNSayVonMBjEdn8>

Experiment 1/3: ESP32C6 Microcontroller

Test-01. Bluetooth

Test-02. HTTP Webservice

Test-03. MQTT

Test-04. CoAP

Test-05. ESP-Now

Test-06. Zigbee

~~Test-07. Thread / Matter~~

~~Test-08. LoRa WAN~~

Test-09. Modbus RTU – Master, Slave

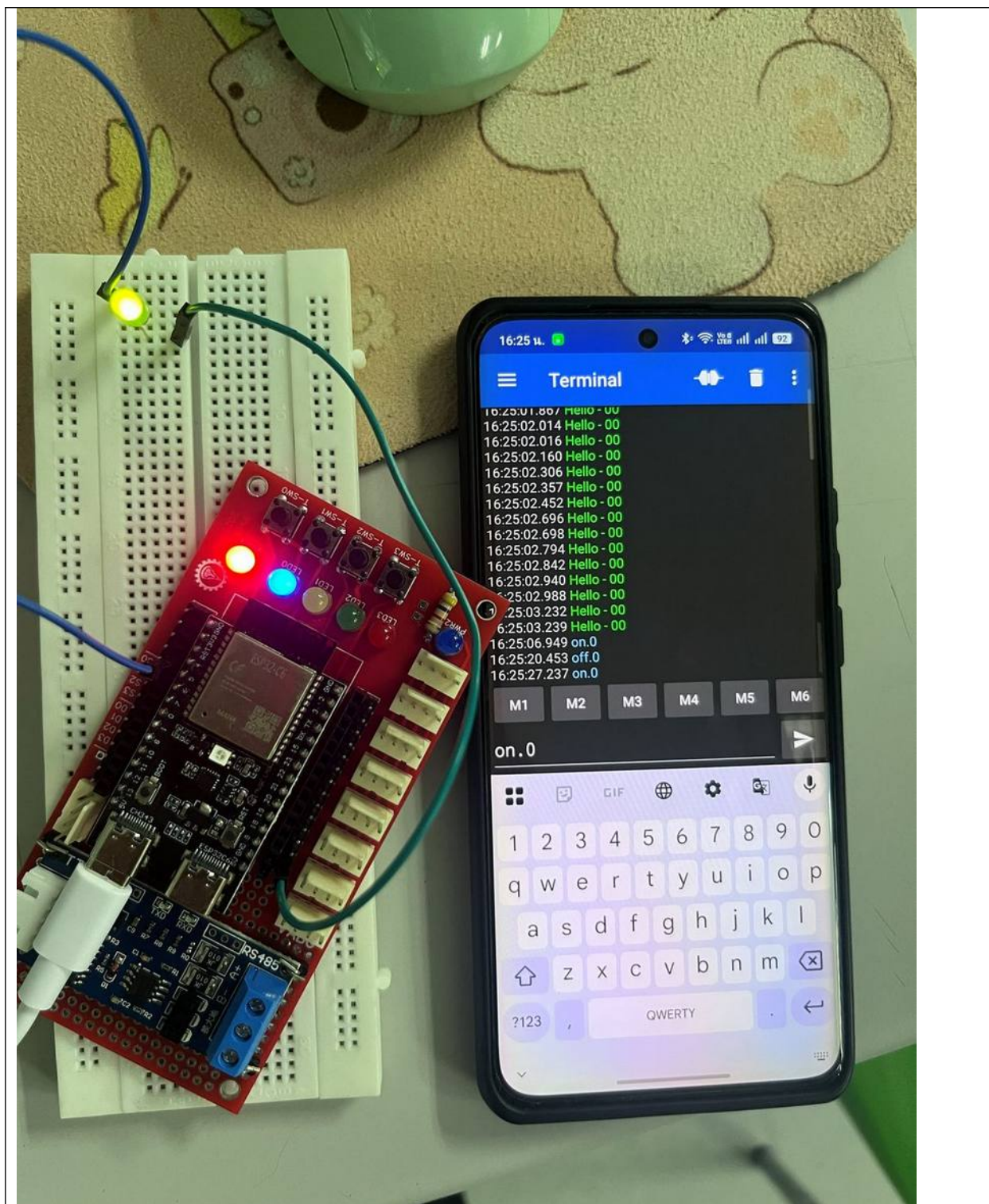
Test-10. Modbus TCP – Server, Client

Test – 01 Bluetooth

The screenshot displays the Arduino IDE interface with a sketch named '01_bluetooth' for an ESP32C6 Dev Module. The sketch includes libraries for BLE and defines pins and UUIDs. The serial monitor shows the boot process and successful BLE initialization.

```
01_bluetooth | Arduino IDE 2.3.7
File Edit Sketch Tools Help
ESP32C6 Dev Module
01_bluetooth.ino
1 #include <BLEDevice.h>
2 #include <BLEServer.h>
3 #include <BLEUtils.h>
4 #include <BLE2902.h>
5
6 // Pin configuration for ESP32-C6 and BLE Name
7 #define myDeviceName "ESP32C6_Test"
8 #define Pin_SMB 1
9 #define Pin_LED0 5
10
11 // Nordic UART Service (NUS) UUIDs
12 #define SERVICE_UUID "6E400001-B5A3-F393-ED09-E50E24DCCAC9"
13 #define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-ED09-E50E24DCCAC9"
14 #define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-ED09-E50E24DCCAC9"
15
16 BLEServer *pServer = NULL;
17 BLECharacteristic *pRxCharacteristic = NULL;
18 bool deviceConnected = false;
19 bool oldDeviceConnected = false;
20
21 // Button Polling state variables
22 bool lastStateSMB = HIGH;
23 unsigned long lastDebounceSMB = 0;
24 const unsigned long debounceDelay = 50; // Debounce threshold in milliseconds
25
26 // Helper function to execute LED commands
27 void controlLED(int pin, const char *name, String action) {
28   if (action == "on") {
29     digitalWrite(pin, HIGH);
30   }
31 }
32
33 Output Serial Monitor X
34 Message (Enter to send message to ESP32C6 Dev Module on COM15)
35 16:05:48.676 -> mode:DBG, clock div:2
36 16:05:48.676 -> load:0x40075730, len:0x1278
37 16:05:48.676 -> load:0x4008b910, len:0x840
38 16:05:48.676 -> load:0x4008e610, len:0x3154
39 16:05:48.676 -> entry 0x4008b910
40 16:05:49.120 -> ESP32-C6 BLZ Ready.
41 16:07:11.524 -> ESP-PCR:esp32c6-20220919
42 16:07:11.524 -> Build: Sep 19 2022
43 16:07:11.565 -> rst:0x1 (POWERON), boot:0x1c (SPI_FLASH_BOOT)
44 16:07:11.565 -> SFDP:0xccc
45 16:07:11.565 -> mode:DBG, clock div:2
46 16:07:11.565 -> load:0x40075730, len:0x1278
47 16:07:11.565 -> load:0x4008b910, len:0x840
48 16:07:11.565 -> load:0x4008e610, len:0x3154
49 16:07:11.565 -> entry 0x4008b910
50 16:07:12.019 -> ESP32-C6 BLZ Ready.
```

Ln 29, Col 24 ESP32C6 Dev Module on COM15 2 16:07 21/8/2566



Test-02. HTTP Webservice

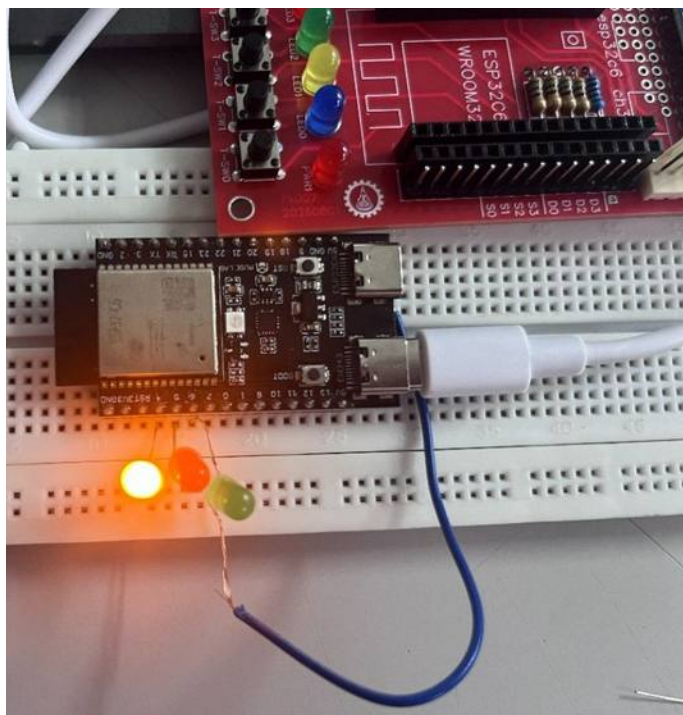
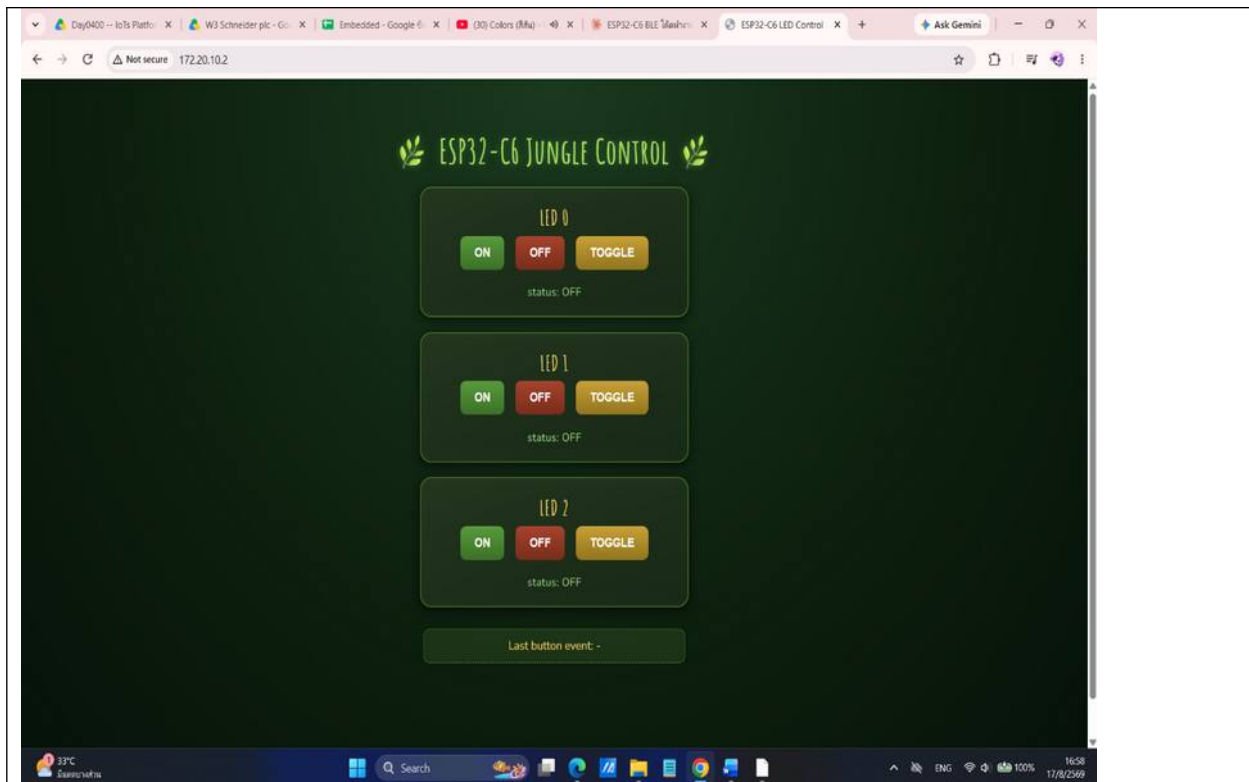
The screenshot displays a web browser window at the address 172.20.10.2, showing a web interface titled "ESP32-C6 JUNGLE CONTROL". The interface features three LED control panels (LED 0, LED 1, LED 2) and a "Last button event" display. Each panel has "ON", "OFF", and "TOGGLE" buttons. The status for LED 0 is "ON", LED 1 is "OFF", and LED 2 is "ON".

Below the browser window, the Arduino IDE (v2.3.7) is open, showing the source code for the "02_Webserver.ino" file. The code is written in C++ and uses the ESP32C6 Dev Module. It includes headers for the Arduino WebServer and HTTP libraries. The code defines a server on port 80 and implements several HTTP handlers: a root handler that serves the index.html file, a toggle handler that toggles the state of a specified LED, and an index handler that lists all LEDs. The code also includes a function to convert a string index to a pin number.

```
190 </script>
191 </body>
192 </html>
193 HTML";
194
195 // ----- HTTP Handlers -----
196 void handleRoot() {
197   server.send(200, "text/html", INDEX_HTML);
198 }
199
200 // /led/<id>/action: on|off|toggle
201 void handleLedCommand() {
202   String url = server.url(); // e.g. "/led/0/on"
203   int firstSlash = url.indexOf('/', 1);
204   int secondSlash = url.indexOf('/', firstSlash + 1);
205
206   if (firstSlash < 0 || secondSlash < 0) {
207     server.send(400, "application/json", "{\"error\":\"bad request\"}");
208     return;
209   }
210
211   int idx = url.substring(firstSlash + 1, secondSlash).toInt();
212   String action = url.substring(secondSlash + 1);
213
214   int pin = ledPinFromIndex(idx);
215   if (pin < 0) {
216     server.send(404, "application/json", "{\"error\":\"invalid led index\"}");
217     return;
218   }
219 }
220
221 void setup() {
222   Serial.begin(115200);
223   Serial.println("ESP32-C6 Web Service Ready");
224 }
225
226 void loop() {
227   server.handleClient();
228 }
```

The Serial Monitor shows the following output:

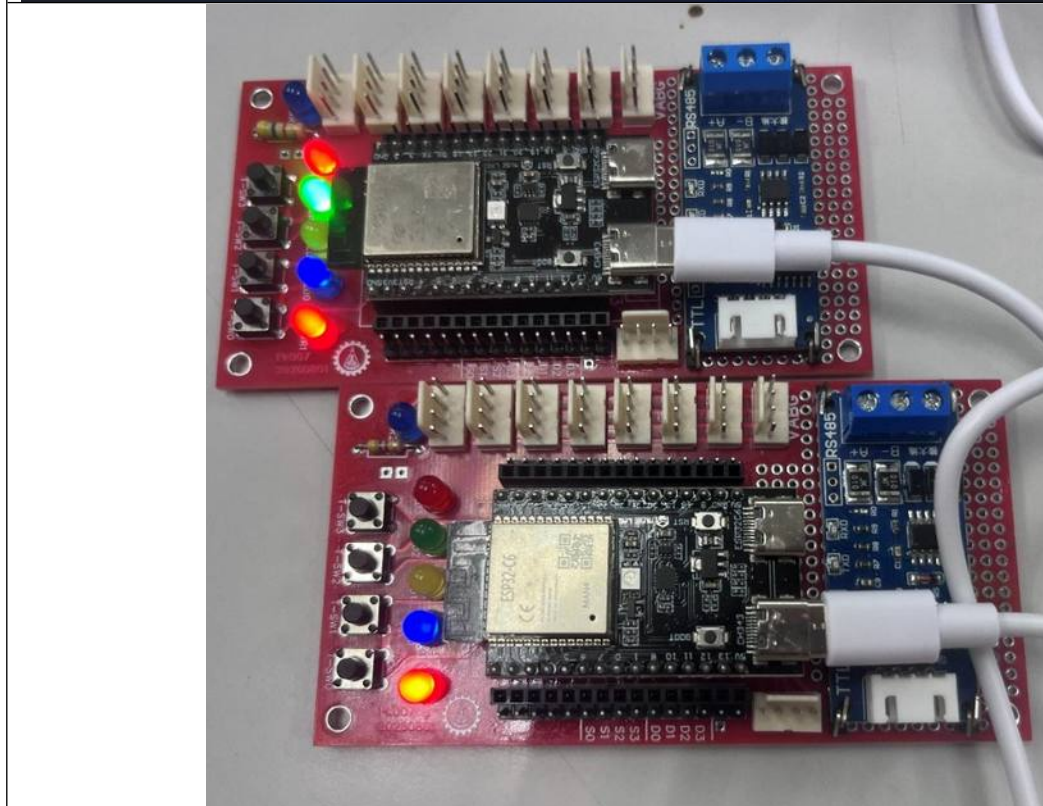
```
17:51:00.654 -> ESP-ROM:esp32c6-20220919
17:51:00.654 -> Boot:Sep 19 2022
17:51:00.654 -> esp-idf (v4.4.0) boot:0x0 (SPI_FLASH_BOOT)
17:51:00.654 -> SPI:0x00000000
17:51:00.654 -> mode:IOIO, eclock:div12
17:51:00.654 -> load:0x40072730, len:0x1278
17:51:00.654 -> load:0x40080910, len:0x400
17:51:00.654 -> load:0x40080e10, len:0x3154
17:51:00.654 -> entry:0x40080910
17:51:00.943 -> Connecting to WiFi...
17:51:02.545 -> Connected! IP address: 172.20.10.2
17:51:02.545 -> ESP32-C6 Web Service Ready
```



Test-03. MQTT

```
03_MQTT | Arduino IDE 2.3.7
File Edit Sketch Tools Help
ESP32C6 Dev Module

03_MQTT.ino
1
2 #include <WiFi.h>
3 #include <PubSubClient.h> // PubSubClient - by Nick O'Leary
4
5 // ===== WiFi Configuration =====
6 const char *ssid = "B";
7 const char *password = "Best158023";
8
9 // ===== MQTT Configuration =====
10 //const char *mqtt_server = "test.mosquitto.org"; // Public broker (d/w/ummaazawda, full auth)
11 const char *mqtt_server = "broker.hivemq.com"; // Public broker d/w/ummaazawda
12 const int mqtt_port = 1883;
13
14 #define myDeviceName "TodsonKai"
15
16 // ----- use 2 topic -----
17 // #007501/esp32c6/control <- (subscribe) d/w/ummaazawda LED
18 // payload (plain text): "dno;action" use "0,on" "1,off" "2,toggle"
19 // #007501/esp32c6/monitor -> (publish) d/w/ummaazawda LED + event (humanId)
20 // payload (plain text): "LED0:1,LED1:0,LED2:1;EVENT:Hello - 00"
21 const char *topicControl = "TodsonKai007/esp32c6/control";
22 const char *topicMonitor = "TodsonKai007/esp32c6/monitor";
23
24 WiFiClient espClient;
25 PubSubClient client(espClient);
26
27 // ===== Pin configuration (ESP32-C6) =====
28 #define Pin_S0 1
29 #define Pin_S0 0
30 #define Pin_S0 7
31
32 Serial Monitor X Output
Message (Enter to send message to ESP32C6 Dev Module on COM15)
New Line 115200 baud
15:57:26.990 -> ESP-ROM:esp32c6-20220919
15:57:26.990 -> Build:Sep 19 2022
15:57:26.990 -> rati0al (POWERON),boot:0ale (SPI_FLASH_BOOT)
15:57:26.990 -> SPIW0:0eee
15:57:26.990 -> mode:000, clock div:2
15:57:26.990 -> load:0x0075730, len:0x1278
15:57:26.990 -> load:0x0080b910, len:0x0d80
15:57:26.990 -> load:0x0080e410, len:0x1154
15:57:26.990 -> entry 0x0080b910
15:57:27.293 -> Connecting to WiFi.....
15:57:29.493 -> Connected! IP address: 10.247.205.6
15:57:29.493 -> Testing TCP connect to broker... SUCCESS
15:57:30.448 -> ESP32-C6 MQTT Service Ready (control/monitor topics, plain text).
15:57:30.648 -> Attempting MQTT connection...
15:57:31.484 -> MQTT connected.
```



Day0409 -- lo x W3 Schneider x Embedded - i x MQTT.Cool ? x (30) Fujii Kazu x How to mqtt x ESP32-C6 BLU x MQTT Webs... x Ask Gemini

hivemq.com/demos/websocket-client/

Websockets Client Showcase

Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.
[Get your free account](#)

Connection

connected

Publish

Topic

TodmonKai007/esp32c6/control

QoS

0

Retain

Message

1,off

Publish

Subscriptions

Add New Topic Subscription

Qos: 0

TodmonKai007/esp3...

Messages

2025-09-17 17:44:23 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:1,LED1:0,LED2:1

2025-09-17 17:44:22 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:1,LED1:0,LED2:1

2025-09-17 17:44:17 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:1,LED1:1,LED2:1

2025-09-17 17:44:10 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:1,LED1:1,LED2:1

2025-09-17 17:44:08 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:1,LED1:1,LED2:1

32°C
Eastermichia

Search

ENG 100% 17:44 17/8/2569

Day0409 -- lo x W3 Schneider x Embedded - i x MQTT.Cool ? x (30) Fujii Kazu x How to mqtt x ESP32-C6 BLU x MQTT Webs... x Ask Gemini

hivemq.com/demos/websocket-client/

Get your own Cloud broker and connect up to 100 devices for free.
[Get your free account](#)

Connection

connected

Publish

Topic

QoS

0

Retain

Message

Publish

Subscriptions

Add New Topic Subscription

Qos: 0

TodmonKai007/esp3...

Messages

2025-09-17 17:40:29 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 02

2025-09-17 17:40:28 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 02

2025-09-17 17:40:28 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 01

2025-09-17 17:40:26 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 01

2025-09-17 17:40:25 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 00

2025-09-17 17:40:22 Topic: TodmonKai007/esp32c6/monitor Qos: 0

LED0:0,LED1:0,LED2:0,EVENT:Hello - 02

32°C
Eastermichia

Search

100% 17:40 17/8/2569

Test-04. CoAP

The image shows two side-by-side Arduino IDE windows. The left window, titled '04_Board1_SentCoAPino', contains the code for a CoAP client (ESP32C6 Dev Module on COM14). The right window, titled '04_Board2_ServerCoAPino', contains the code for a CoAP server (ESP32C6 Dev Module on COM15). Both windows show the Serial Monitor output at the bottom.

Left Window: 04_Board1_SentCoAPino

```
1 #include <WiFi.h>
2 #include <coap-simple.h>
3
4 #define WIFI_SSID "Softnet"
5 #define WIFI_PASSWORD "12345667"
6
7 WiFiUDP udp;
8 Coap coap(udp);
9
10 //con port: 5683
11 void callback_response(CoapPacket ipacket, IPAddress ip, int port) {
12   Serial.println("Response received from CoAP Server");
13 }
14
15 void setup() {
16   Serial.begin(115200);
17   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
18
19   while (WiFi.status() != WL_CONNECTED) {
20     delay(500);
21     Serial.print(".");
22   }
23
24   Serial.println("connected to WiFi");
25   coap.start();
26   coap.response(callback_response);
27 }
28
29 void loop() {
30   float temp = random(5000) / 100.0;
31   float humid = random(5000) / 100.0 + 50;
32
33   if (!isnan(temp) && !isnan(humid)) {
34     String payload = "{\"temperature\":\"" + String(temp) + "\", \"humidity\":\"" + String(humid) + "\"}";
35     coap.send("/sensor/data/177.50.10.11.5683. \"sensor/data\". payload: " + payload);
36   }
37 }
```

Right Window: 04_Board2_ServerCoAPino

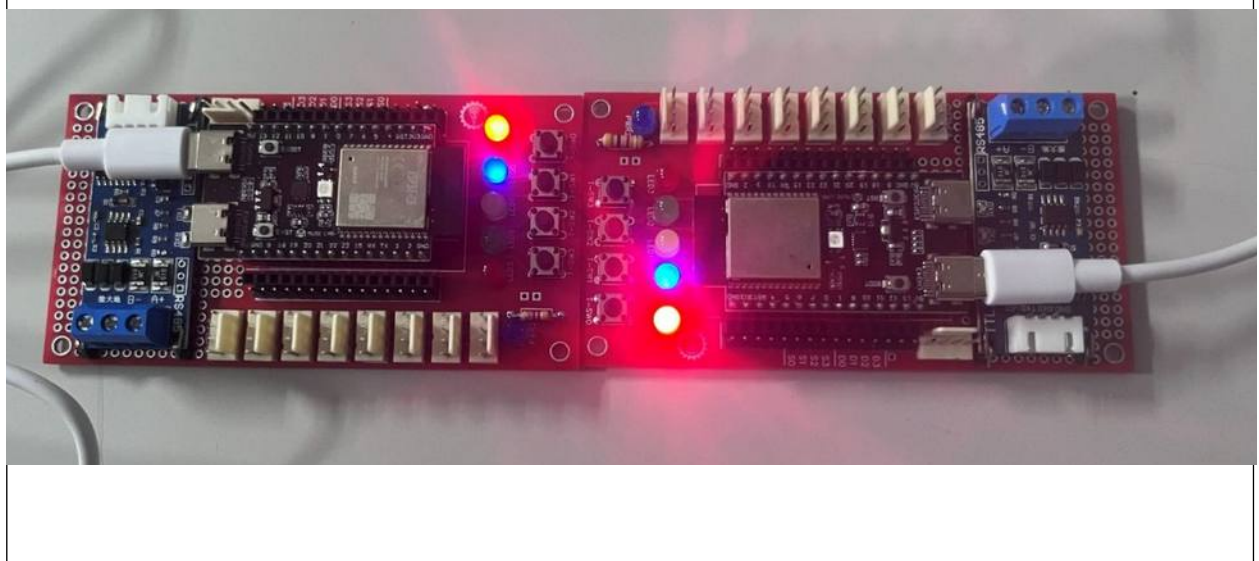
```
15 payload[payload.length] = '\0';
16
17 Serial.print("Received from ");
18 Serial.println(ip);
19 Serial.print("port:");
20 Serial.println(port);
21 Serial.print(" -> ");
22 Serial.println(payload);
23
24 // maundu client 317bauds (2.84 Changed)
25 coap.sendResponse(ip, port, packet.messageId);
26
27
28 void setup() {
29   Serial.begin(115200);
30   WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
31
32   Serial.print("Connecting to WiFi");
33   while (WiFi.status() != WL_CONNECTED) {
34     delay(500);
35     Serial.print(".");
36   }
37
38   Serial.println();
39   Serial.print("Connected IP address: ");
40   Serial.println(WiFi.localIP());
41
42   // ip resource "sensor/data" và callback
43   coap.server(callback_sensor_data, "sensor/data");
44
45   coap.start();
46   Serial.println("CoAP Server Ready. Waiting for data...");
47 }
48
49 void loop() {
50   coap.loop();
51 }
```

Serial Monitor Output (Left):

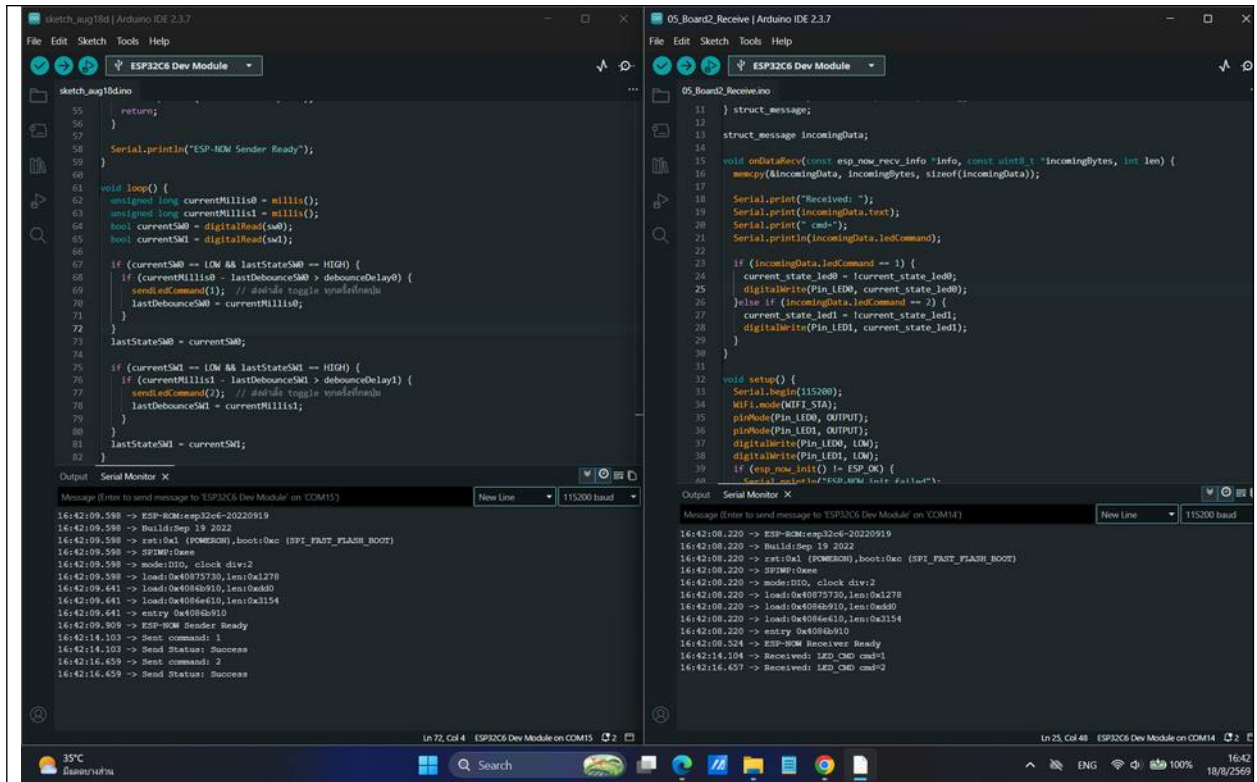
```
15:39:42.293 -> Data Sent: {"temperature":26.41, "humidity":77.16}
15:39:42.293 -> Response received from CoAP Server
15:39:47.283 -> Data Sent: {"temperature":42.17, "humidity":79.13}
15:39:47.283 -> Response received from CoAP Server
15:39:52.248 -> Data Sent: {"temperature":42.07, "humidity":69.76}
15:39:52.250 -> Response received from CoAP Server
```

Serial Monitor Output (Right):

```
15:39:37.405 -> Received from 172.20.10.2:5683 -> {"temperature":44.45, "humidity":50.45}
15:39:42.330 -> Received from 172.20.10.2:5683 -> {"temperature":26.41, "humidity":77.16}
15:39:47.318 -> Received from 172.20.10.2:5683 -> {"temperature":42.17, "humidity":79.13}
15:39:52.410 -> Received from 172.20.10.2:5683 -> {"temperature":42.07, "humidity":69.76}
```



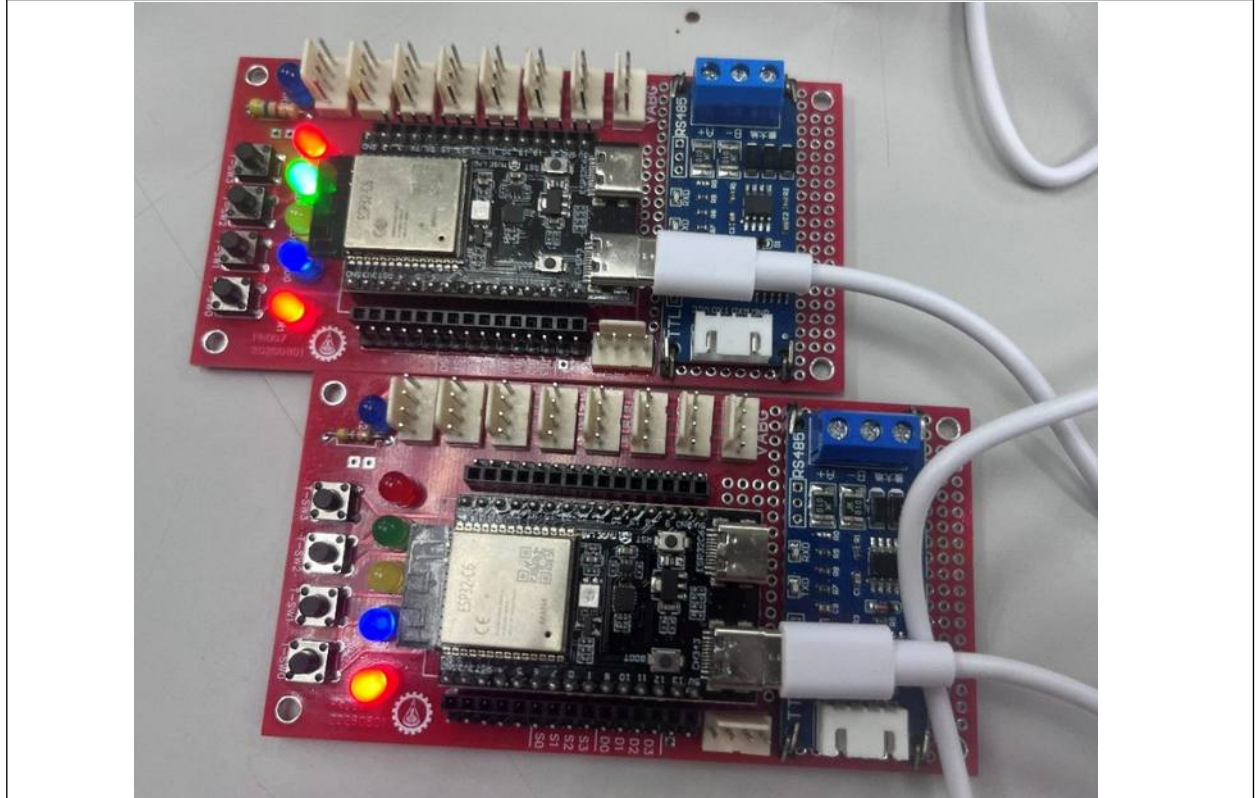
Test-05. ESP-Now



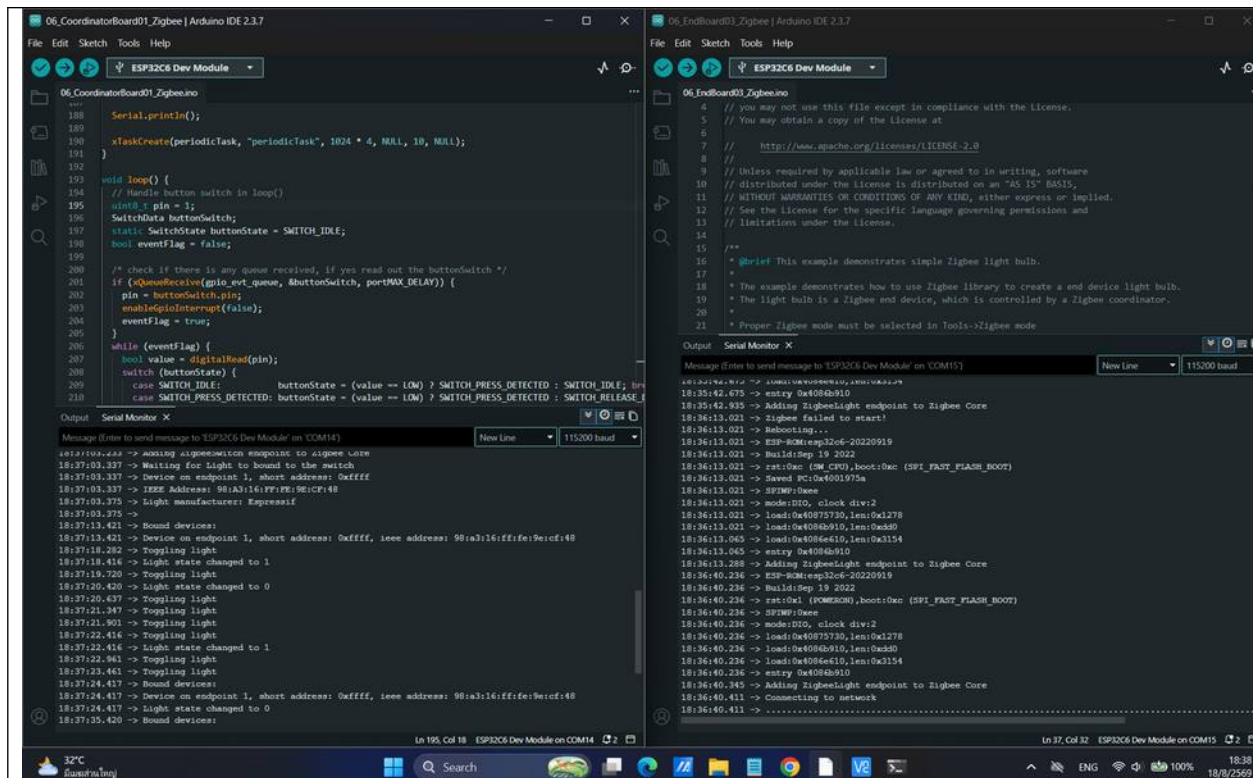
The image shows two side-by-side Arduino IDE windows. The left window, titled 'sketch_aug18d', contains the sender code for an ESP-Now module. It includes a setup function for serial communication and a loop function that checks for button presses on pins 5 and 6, sending commands 1 and 2 via ESP-Now. The right window, titled '05_Board2_Receive', contains the receiver code. It defines a message structure, sets up pins 13 and 14 for LEDs, and implements a receive function that prints received data and toggles the LEDs based on the received command. Both windows have their serial monitors open, showing the transmission of commands and the receipt of data.

```
sketch_aug18d
File Edit Sketch Tools Help
ESP32C6 Dev Module
sketch_aug18dino
55 return;
56 }
57 Serial.println("ESP-NOW Sender Ready");
58 }
59
60 void loop() {
61   unsigned long currentMillis0 = millis();
62   unsigned long currentMillis1 = millis();
63   bool currentSMB = digitalRead(sw0);
64   bool currentSM1 = digitalRead(sw1);
65
66   if (currentSMB == LOW && lastStateSMB == HIGH) {
67     if (currentMillis0 - lastDebounceSMB > debounceDelay0) {
68       sendCmdCommand(1); // send toggle vspndelnsu
69       lastDebounceSMB = currentMillis0;
70     }
71   }
72   lastStateSMB = currentSMB;
73
74   if (currentSM1 == LOW && lastStateSM1 == HIGH) {
75     if (currentMillis1 - lastDebounceSM1 > debounceDelay1) {
76       sendCmdCommand(2); // send toggle vspndelnsu
77       lastDebounceSM1 = currentMillis1;
78     }
79   }
80   lastStateSM1 = currentSM1;
81 }
82
Output Serial Monitor X
Message (Enter to send message to ESP32C6 Dev Module on COM15)
New Line 115200 baud
16:42:09.598 -> ESP-NOW:esp32c6-20220919
16:42:09.598 -> BuildID: 19 2022
16:42:09.598 -> rsi:0x1 (POM2020),boot:0x0 (SPI_FAST_FLASH_BOOT)
16:42:09.598 -> SPIWP:0xxx
16:42:09.598 -> mode:DI0, clock div:2
16:42:09.598 -> load:0x40073730,len:0x1278
16:42:09.641 -> load:0x4008b910,len:0xd8d0
16:42:09.641 -> load:0x4008e610,len:0x3154
16:42:09.641 -> entry 0x4008b910
16:42:09.909 -> ESP-NOW Sender Ready
16:42:14.103 -> Send command: 1
16:42:14.103 -> Send Status: Success
16:42:16.659 -> Send command: 2
16:42:16.659 -> Send Status: Success

05_Board2_Receive
File Edit Sketch Tools Help
ESP32C6 Dev Module
05_Board2_Receiveino
11 struct message;
12
13 struct message incomingData;
14
15 void onDataRecv(const esp_now_rec_info *info, const uint8_t *incomingBytes, int len) {
16   memcpy(&incomingData, incomingBytes, sizeof(incomingData));
17
18   Serial.print("Received: ");
19   Serial.println(incomingData.text);
20   Serial.print(" cmd-");
21   Serial.println(incomingData.cmdCommand);
22
23   if (incomingData.cmdCommand == 1) {
24     current_state_led0 = !current_state_led0;
25     digitalWrite(Pin_LED0, current_state_led0);
26   } else if (incomingData.cmdCommand == 2) {
27     current_state_led1 = !current_state_led1;
28     digitalWrite(Pin_LED1, current_state_led1);
29   }
30 }
31
32 void setup() {
33   Serial.begin(115200);
34   pinMode(MIFI_STA);
35   pinMode(Pin_LED0, OUTPUT);
36   pinMode(Pin_LED1, OUTPUT);
37   digitalWrite(Pin_LED0, LOW);
38   digitalWrite(Pin_LED1, LOW);
39   if (esp_now_init() != ESP_OK) {
40     Serial.println("ESP-NOW init failed!");
41   }
42 }
43
Output Serial Monitor X
Message (Enter to send message to ESP32C6 Dev Module on COM14)
New Line 115200 baud
16:42:08.220 -> ESP-NOW:esp32c6-20220919
16:42:08.220 -> BuildID: 19 2022
16:42:08.220 -> rsi:0x1 (POM2020),boot:0x0 (SPI_FAST_FLASH_BOOT)
16:42:08.220 -> SPIWP:0xxx
16:42:08.220 -> mode:DI0, clock div:2
16:42:08.220 -> load:0x40073730,len:0x1278
16:42:08.220 -> load:0x4008b910,len:0xd8d0
16:42:08.220 -> load:0x4008e610,len:0x3154
16:42:08.220 -> entry 0x4008b910
16:42:08.524 -> ESP-NOW Receiver Ready
16:42:14.104 -> Received: LED_CMD cmd=1
16:42:16.657 -> Received: LED_CMD cmd=2
```



Test-06. Zigbee



The image shows two side-by-side Arduino IDE windows. The left window, titled '06_CoordinatorBoard01_Zigbee', contains the code for a Zigbee coordinator. It includes a serial monitor output showing the device booting and joining the network. The right window, titled '06_EndBoard03_Zigbee', contains the code for a Zigbee end device (light bulb). It includes a serial monitor output showing the device booting and connecting to the network. Both windows show the 'Serial Monitor' window with the output of the code execution.

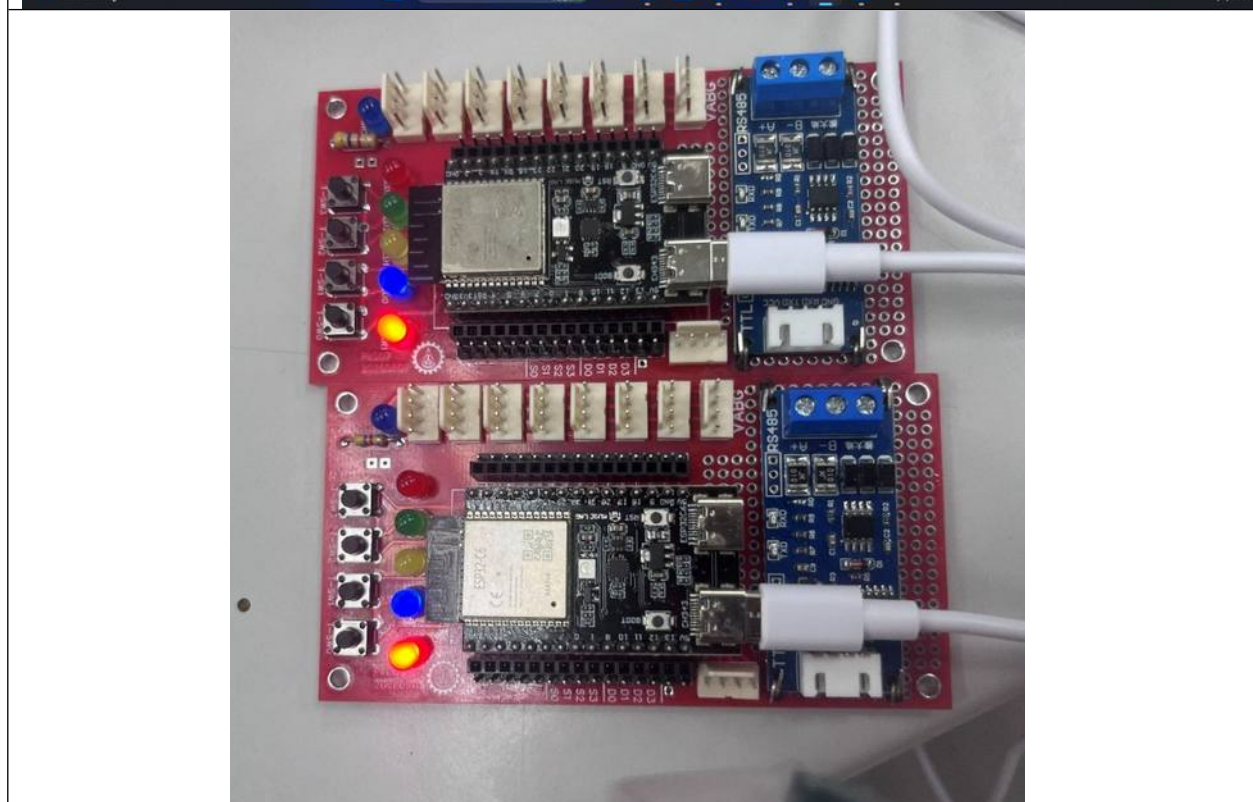
```
06_CoordinatorBoard01_Zigbee.ino
188 Serial.println();
189
190 xTaskCreate(PeriodicTask, "PeriodicTask", 1024 * 4, NULL, 10, NULL);
191 }
192
193 void loop() {
194   // Handle button switch in loop()
195   uint8_t pin = 1;
196   SwitchData buttonSwitch;
197   static SwitchState buttonState = SWITCH_IDLE;
198   bool eventFlag = false;
199
200   /* check if there is any queue received, if yes read out the buttonSwitch */
201   if (xQueueReceive(gpio_evt_queue, &buttonSwitch, portMAX_DELAY)) {
202     pin = buttonSwitch.pin;
203     enableGpioInterrupt(false);
204     eventFlag = true;
205   }
206
207   while (eventFlag) {
208     bool value = digitalRead(pin);
209     switch (buttonState) {
210       case SWITCH_IDLE:
211         buttonState = (value == LOW) ? SWITCH_PRESS_DETECTED : SWITCH_IDLE; break;
212       case SWITCH_PRESS_DETECTED:
213         buttonState = (value == LOW) ? SWITCH_PRESS_DETECTED : SWITCH_RELEASE_DETECTED; break;
214     }
215   }
216 }
```

```
06_EndBoard03_Zigbee.ino
4 // You may not use this file except in compliance with the License.
5 // You may obtain a copy of the License at
6 //
7 // http://www.apache.org/licenses/LICENSE-2.0
8 //
9 // Unless required by applicable law or agreed to in writing, software
10 // distributed under the License is distributed on an "AS IS" BASIS,
11 // WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
12 // See the license for the specific language governing permissions and
13 // limitations under the License.
14
15 /**
16  * @brief This example demonstrates simple Zigbee light bulb.
17  *
18  * The example demonstrates how to use Zigbee library to create a end device light bulb.
19  * The light bulb is a Zigbee end device, which is controlled by a Zigbee coordinator.
20  * Proper Zigbee mode must be selected in Tools->Zigbee mode
21  */
22
```

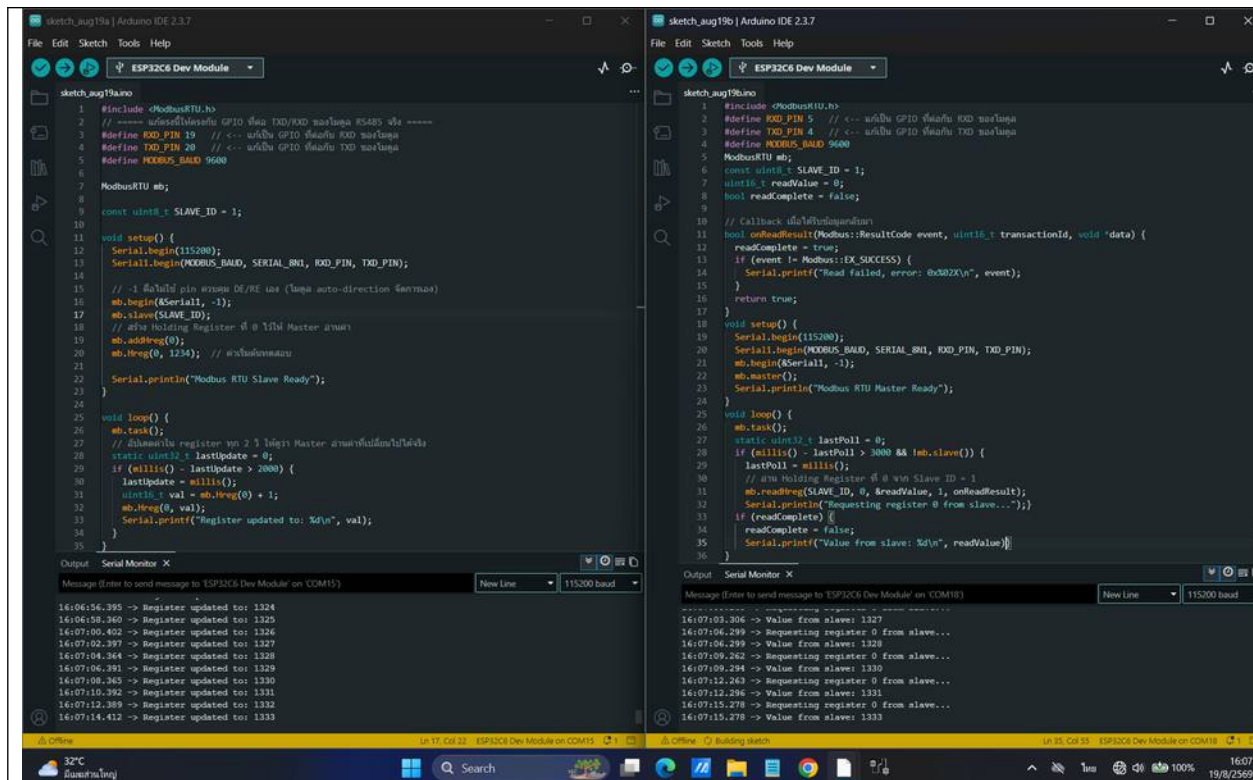
Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on COM15)

18:36:40.411 -> entry 0x4084b910
18:36:40.411 -> Adding Zigbee light endpoint to Zigbee Core
18:36:40.411 -> Zigbee failed to start!
18:36:40.411 -> Rebooting...
18:36:40.411 -> ESP-ROM:esp32c6-20220919
18:36:40.411 -> Build:Sep 19 2022
18:36:40.411 -> rdt:0x1 (POWERON), boot:0x0 (SPI_FAST_FLASH_BOOT)
18:36:40.411 -> Saved PC:0x40001975a
18:36:40.411 -> SPIW0:0x0
18:36:40.411 -> mode:DI0, clock div:2
18:36:40.411 -> load:0x40075730, len:0x1278
18:36:40.411 -> load:0x40084b910, len:0x4080
18:36:40.411 -> load:0x40084b910, len:0x154
18:36:40.411 -> entry 0x4084b910
18:36:40.411 -> Adding Zigbee light endpoint to Zigbee Core
18:36:40.411 -> ESP-ROM:esp32c6-20220919
18:36:40.411 -> Build:Sep 19 2022
18:36:40.411 -> rdt:0x1 (POWERON), boot:0x0 (SPI_FAST_FLASH_BOOT)
18:36:40.411 -> SPIW0:0x0
18:36:40.411 -> mode:DI0, clock div:2
18:36:40.411 -> load:0x40075730, len:0x1278
18:36:40.411 -> load:0x40084b910, len:0x4080
18:36:40.411 -> load:0x40084b910, len:0x154
18:36:40.411 -> entry 0x4084b910
18:36:40.411 -> Adding Zigbee light endpoint to Zigbee Core
18:36:40.411 -> Connecting to network
18:36:40.411 ->



Test-09. Modbus RTU – Master, Slave



```
1 #include <ModbusRTU.h>
2 // ----- กำหนดค่าพิน GPIO ที่ใช้ TXD/RXD มาทำงาน RS485 -----
3 #define RXD_PIN 5 // <-> ใช้เป็น GPIO ที่ทำหน้าที่ RXD มาทำงาน
4 #define TXD_PIN 4 // <-> ใช้เป็น GPIO ที่ทำหน้าที่ TXD มาทำงาน
5 #define MODBUS_BAUD 9600
6
7 ModbusRTU mb;
8
9 const uint8_t SLAVE_ID = 1;
10
11 void setup() {
12   Serial.begin(115200);
13   Serial1.begin(MODBUS_BAUD, SERIAL_8N1, RXD_PIN, TXD_PIN);
14
15   // -1 ทำหน้าที่ pin ควบคุม DE/RE ใช้ (ทำงาน auto-direction ง่ายมาก)
16   mb.begin(&Serial1, -1);
17   mb.slave(SLAVE_ID);
18   // ถ้า Holding Register 0 ของ Slave Master อ่านได้
19   mb.writeReg(0); // ทำให้อ่านค่าได้
20   mb.readReg(0, 1234); // ทำให้อ่านค่าได้
21
22   Serial.println("Modbus RTU Slave Ready");
23 }
24
25 void loop() {
26   mb.task();
27
28   // ถ้า Holding Register 0 ของ Master Master อ่านได้
29   static uint32_t lastUpdate = 0;
30   if (millis() - lastUpdate > 2000) {
31     lastUpdate = millis();
32     uint16_t val = mb.readReg(0) + 1;
33     mb.writeReg(0, val);
34     Serial.printf("Register updated to: %d\n", val);
35   }
36 }
```

```
1 #include <ModbusRTU.h>
2 #define RXD_PIN 5 // <-> ใช้เป็น GPIO ที่ทำหน้าที่ RXD มาทำงาน
3 #define TXD_PIN 4 // <-> ใช้เป็น GPIO ที่ทำหน้าที่ TXD มาทำงาน
4 #define MODBUS_BAUD 9600
5
6 ModbusRTU mb;
7
8 const uint8_t SLAVE_ID = 1;
9
10 uint16_t readValue = 0;
11 bool readComplete = false;
12
13 // Callback เมื่อได้รับข้อมูล
14 bool onReadResult(Modbus::ResultCode event, uint16_t transactionId, void *data) {
15   readComplete = true;
16   if (event != Modbus::EX_SUCCESS) {
17     Serial.printf("Read failed, error: %d\n", event);
18   }
19   return true;
20 }
21
22 void setup() {
23   Serial.begin(115200);
24   Serial1.begin(MODBUS_BAUD, SERIAL_8N1, RXD_PIN, TXD_PIN);
25   mb.begin(&Serial1, -1);
26   mb.master();
27   Serial.println("Modbus RTU Master Ready");
28 }
29
30 void loop() {
31   mb.task();
32
33   static uint32_t lastPoll = 0;
34   if (millis() - lastPoll > 3000 && !mb.slave()) {
35     lastPoll = millis();
36     // ถ้า Holding Register 0 ของ Slave ID = 1
37     mb.readReg(SLAVE_ID, 0, &readValue, 1, onReadResult);
38     Serial.printf("Requesting register 0 from slave...\n");
39     if (readComplete) {
40       readComplete = false;
41       Serial.printf("Value from slave: %d\n", readValue);
42     }
43   }
44 }
```

Output Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on COM15)

New Line 115200 baud

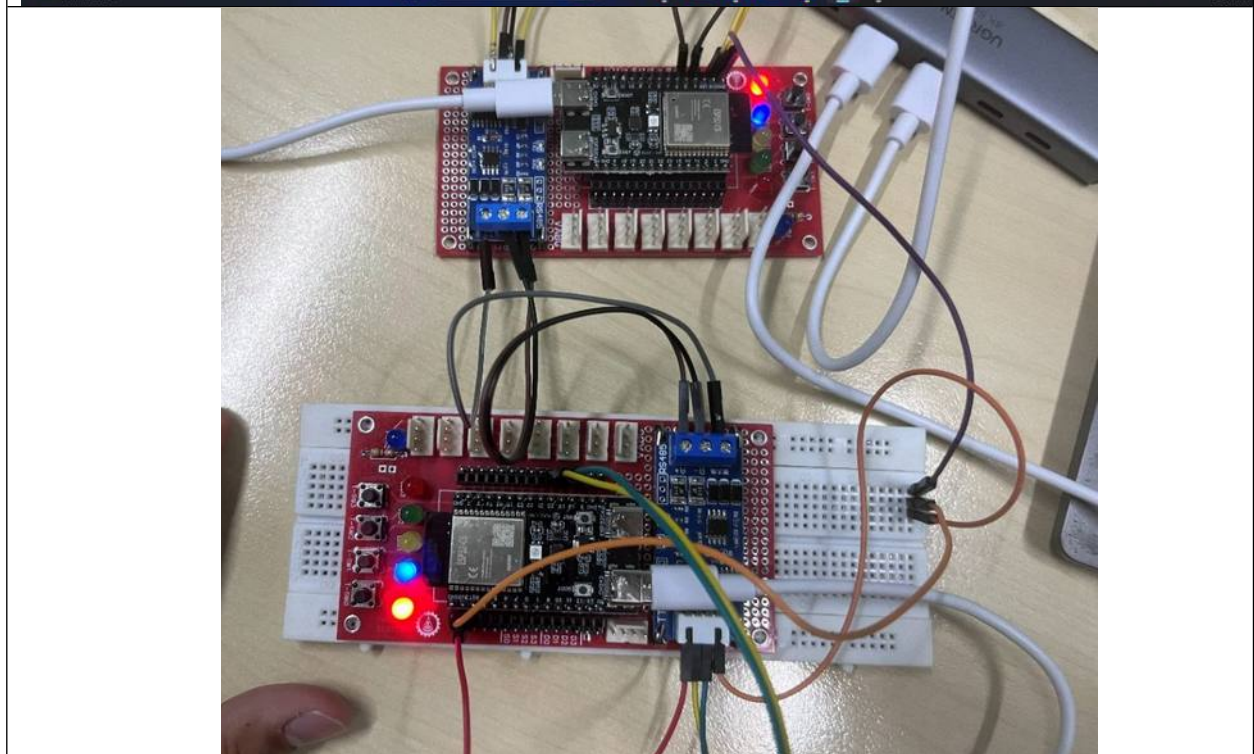
16:06:56.395 -> Register updated to: 1324
16:06:58.360 -> Register updated to: 1325
16:07:00.402 -> Register updated to: 1326
16:07:02.397 -> Register updated to: 1327
16:07:04.364 -> Register updated to: 1328
16:07:06.391 -> Register updated to: 1329
16:07:08.363 -> Register updated to: 1330
16:07:10.392 -> Register updated to: 1331
16:07:12.385 -> Register updated to: 1332
16:07:14.412 -> Register updated to: 1333

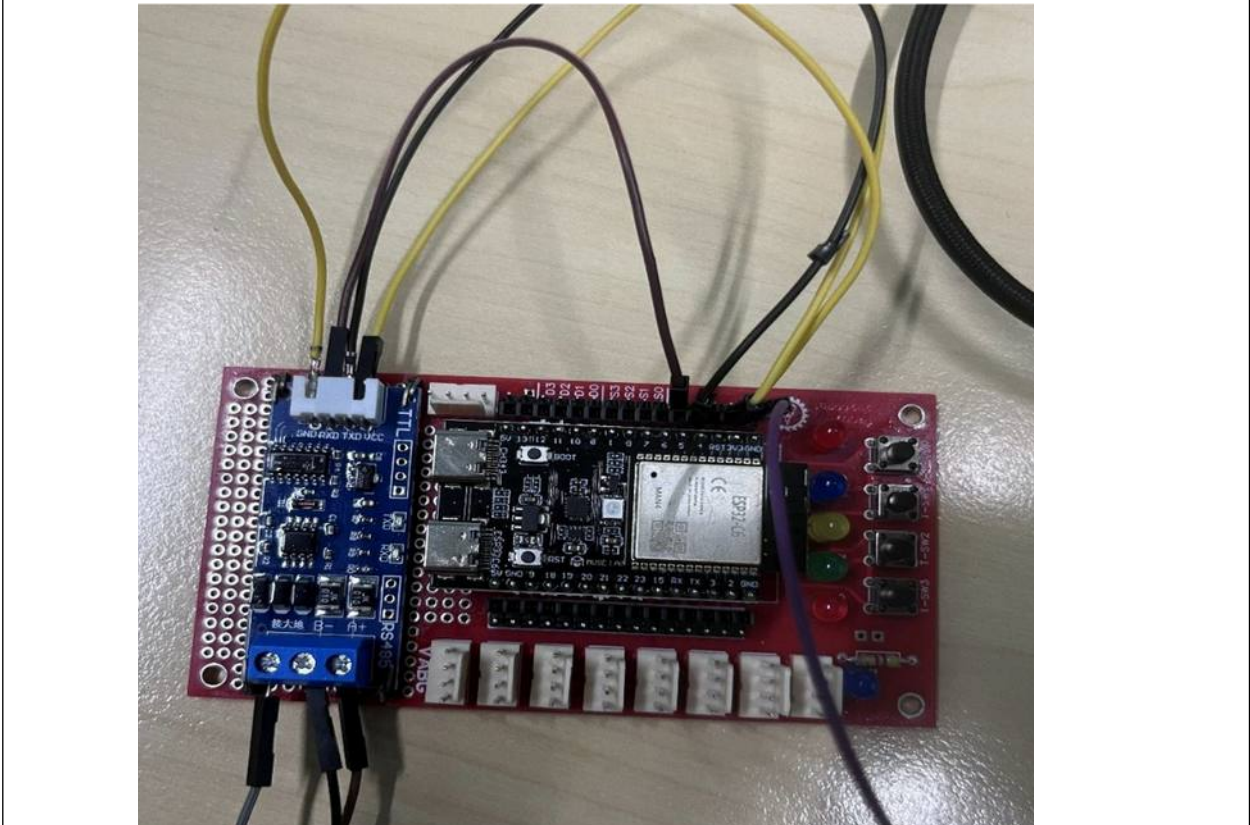
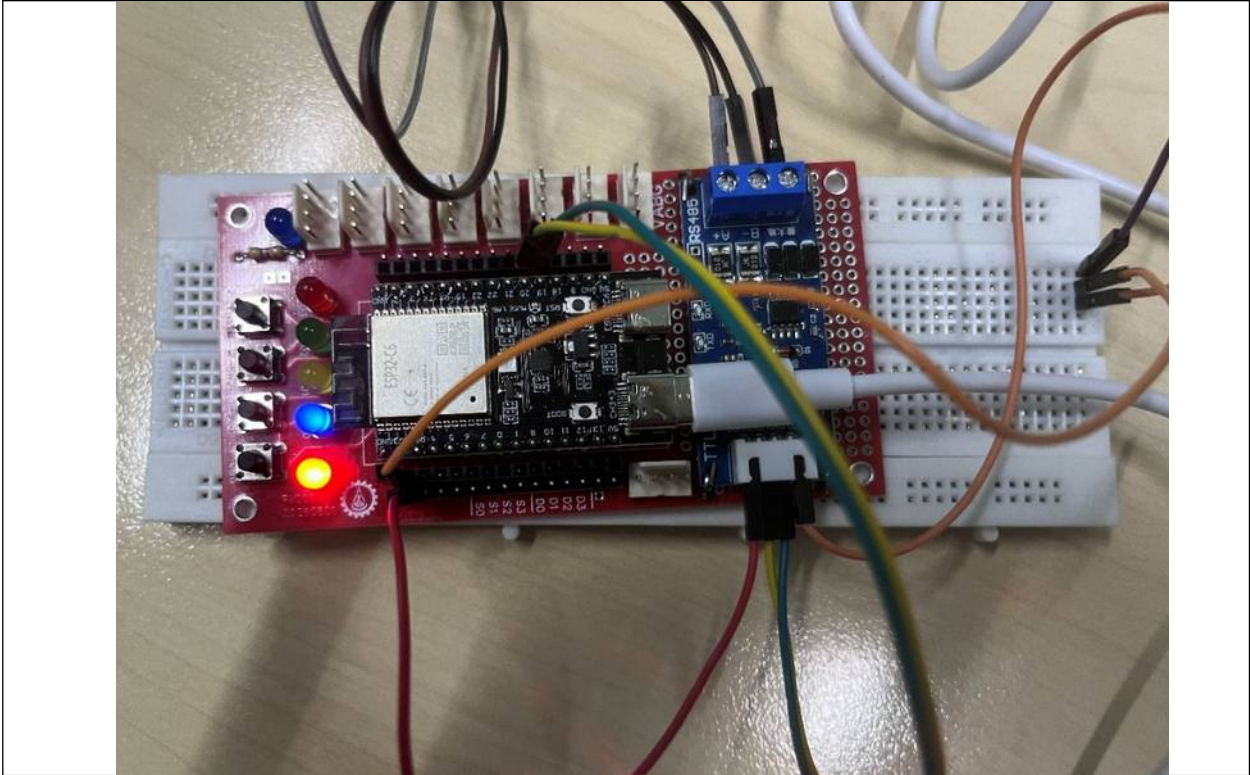
Output Serial Monitor X

Message (Enter to send message to 'ESP32C6 Dev Module' on COM18)

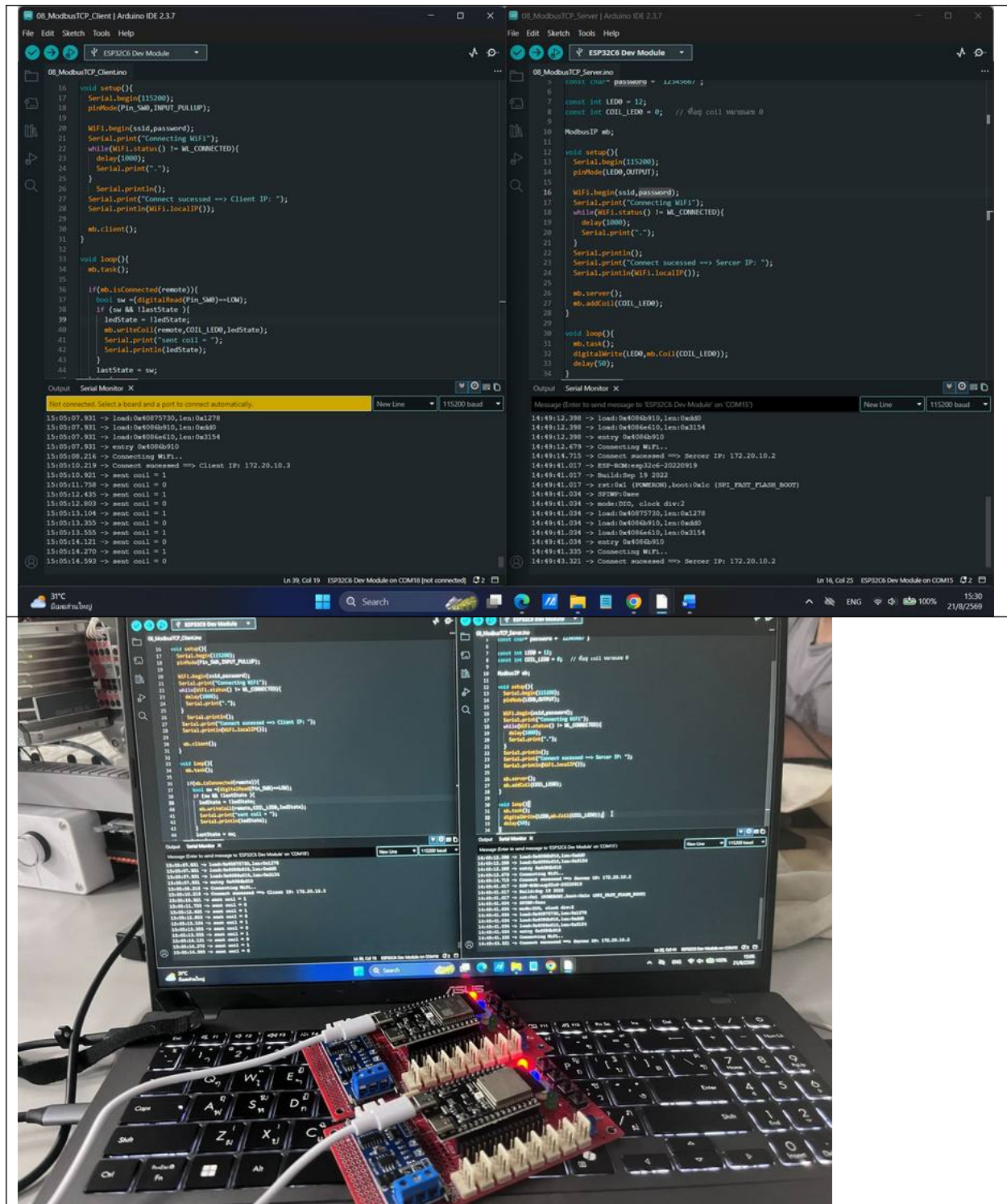
New Line 115200 baud

16:07:03.306 -> Value from slave: 1327
16:07:04.299 -> Requesting register 0 from slave...
16:07:04.299 -> Value from slave: 1328
16:07:09.262 -> Requesting register 0 from slave...
16:07:09.294 -> Value from slave: 1330
16:07:12.263 -> Requesting register 0 from slave...
16:07:12.296 -> Value from slave: 1331
16:07:15.271 -> Requesting register 0 from slave...
16:07:15.278 -> Value from slave: 1333





Test-10. Modbus TCP – Server, Client



eriment 2/3: ESPHome

How ESPHome works

Building your own smart home device from scratch can feel daunting, particularly if you're not familiar with coding. Not with ESPHome. Its intuitive Device Builder replaces code with simple visual choices for a hassle-free setup that brings you straight into building without touching a command line. And for experienced users who prefer writing code manually, YAML is always there if you need it. Every device works out of the box with Home Assistant, too – locally, privately, and securely.

Step 1: Choose & configure: Select your board and components using the visual Device Builder or a YAML file. Pick sensors from the catalog and set your options without writing code.

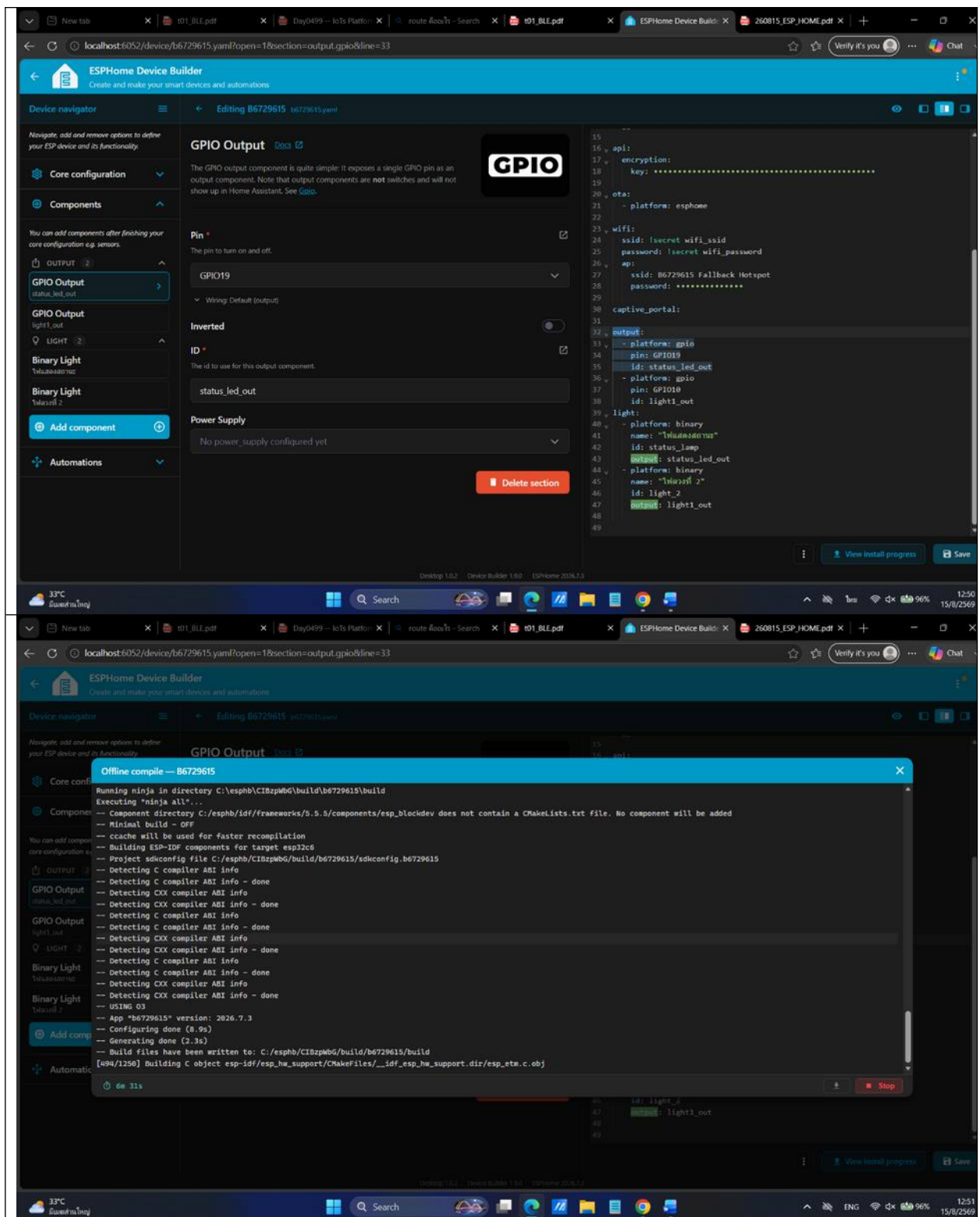
Step 2: Build & install: ESPHome installs your configuration to your device straight from your browser, or wirelessly over the air.

Step 3: Connect & control: Your new device automatically pops up in Home Assistant over your local network: no cloud accounts or external servers required.

Read More

- <https://esphome.io/>

ESP HOME



ESPHome Device Builder

GPIO Output

Logs - B6729615 (Web Serial)

```
[13:27:07][C][api:270]: Noise encryption: YES
[13:27:07][C][mdns:250]: mDNS:
[13:27:07][C][mdns:250]: Hostname: b6729615
[13:27:10][O][wifi:1485]: Found networks:
[13:27:10][I][wifi:1486]: ~ 'SonTua' (8A:49:AF:3A:9F:A1) Ch: 6 -53dB P:0
[13:27:10][O][wifi:1482]: Retry phase: INITIAL_CONNECT + SCAN_CONNECTING
[13:27:10][I][wifi:1119]: Connecting to 'SonTua' (8A:49:AF:3A:9F:A1) (priority 0, attempt 1/2 in phase SCAN_CONNECTING)...
[13:27:11][I][wifi:1698]: Connected
[13:27:11][O][wifi:1617]: Disabling AP
[13:27:11][C][wifi:1209]: IP Address: 172.20.10.3
[13:27:11][C][wifi:1270]: SSID: 'SonTua'
[13:27:11][C][wifi:1270]: BSSID: 8A:49:AF:3A:9F:A1
[13:27:11][C][wifi:1270]: Hostname: 'b6729615'
[13:27:11][C][wifi:1270]: Signal strength: -57 dB
[13:27:11][C][wifi:1270]: Channel: 6
[13:27:11][C][wifi:1270]: Subnet: 255.255.255.240
[13:27:11][C][wifi:1270]: Gateway: 172.20.10.1
[13:27:11][C][wifi:1270]: DNS1: 172.20.10.1
[13:27:11][C][wifi:1270]: DNS2: 0.0.0.0
[13:27:11][W][component:365]: wifi cleared warning flag
[13:28:07][I][safe_mode:142]: Boot seems successful; resetting boot loop counter
[13:28:11][O][preferences:245]: Writing 1 items: 0 cached, 1 written, 0 failed
[13:32:11][O][wifi:2480]: Roam scan (-54 dBm, attempt 1/3)
```

No serial output from the device yet. [Stream network logs instead](#)

platform: wifi_info
ip_address1
name: wurman10d
icon: "wifi"

13°C
15/8/2024

Experiment 3/3: ESP32C6 and AI

แนวทางการนำ LLM มาใช้บน ESP32-C6

Read More

- Running a 28.9M parameter LLM on a microcontroller
<https://github.com/slvDev/esp32-ai>
- <https://www.facebook.com/photo?fbid=1691033469697857&set=pb.100063736613709.-2207520000>